Week 2 Lab: Diffusion Models (DDPM & DDIM)

Exercise 1

(a) In one or two sentences each, describe the *forward process* and the *reverse process* of a diffusion model. Which of the two contains learnable parameters?
(b) The lecture's "denoise" block does not output a clean image directly; it contains a *noise predictor*. What does the noise predictor receive as input, and what does it output?
(c) The same network is used for all 1000 denoising steps. Why must the step index t be given to it as an input?

Solution

(a) Forward: a fixed (hand-designed) procedure that gradually adds Gaussian noise to a data point x_0 over T steps until only noise $x_T \approx \mathcal{N}(0, I)$ remains. Reverse: starting from pure noise, repeatedly remove a little noise until an image appears. Only the reverse process is learned; the forward process has no parameters.
(b) Input: the noisy image x_t and the step t (plus a condition such as a text embedding in text-to-image models). Output: an estimate $\varepsilon_\theta(x_t, t)$ of the noise contained in x_t. The denoise step then subtracts (a scaled version of) this estimate.
(c) The amount of noise differs strongly between steps (almost none at $t = 1$, almost pure noise at $t = 1000$). The network must know how noisy its input is to know how much to remove and what kind of structure to predict: coarse layout at large t, fine detail at small t. Sharing one network across steps saves parameters, and the t-embedding lets it behave differently per step.

Exercise 2

The forward process adds noise one step at a time: $x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon_t$, with independent $\varepsilon_t \sim \mathcal{N}(0, I)$.

(a) Show that two steps can be written as a single step, $x_2 = \sqrt{\bar{\alpha}_2}\, x_0 + \sqrt{1-\bar{\alpha}_2}\, \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, I)$. (Use: the sum of two independent zero-mean Gaussians with variances a and b is a zero-mean Gaussian with variance $a + b$.)
(b) State the general result for x_t and explain in words why it matters for *training*.

Solution

(a) $x_1 = \sqrt{\alpha_1} x_0 + \sqrt{1-\alpha_1}\varepsilon_1$, so $x_2 = \sqrt{\alpha_2}(\sqrt{\alpha_1} x_0 + \sqrt{1-\alpha_1}\varepsilon_1) + \sqrt{1-\alpha_2}\varepsilon_2 = \sqrt{\alpha_1\alpha_2}\, x_0 + \underbrace{\sqrt{\alpha_2(1-\alpha_1)}\, \varepsilon_1 + \sqrt{1-\alpha_2}\, \varepsilon_2}_{\text{variance } \alpha_2 - \alpha_1\alpha_2 + 1 - \alpha_2 = 1 - \alpha_1\alpha_2}$. So $x_2 = \sqrt{\bar{\alpha}_2}\, x_0 + \sqrt{1-\bar{\alpha}_2}\, \varepsilon$.
(b) By induction, $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon$. For training we can create a noisy image at *any* step directly from x_0 with one noise sample, instead of simulating t sequential steps ("ideally … in reality", p. 15). This makes each training iteration cheap and lets us sample t at random.

Exercise 3

DDPM uses a linear schedule with $\beta_1 = 10^{-4}$ rising to $\beta_{1000} = 0.02$. For this schedule, $\bar{\alpha}_{10} \approx 0.998$, $\bar{\alpha}_{100} \approx 0.897$, $\bar{\alpha}_{250} \approx 0.524$, $\bar{\alpha}_{500} \approx 0.079$ and $\bar{\alpha}_{1000} \approx 4 \times 10^{-5}$.

(a) For each t, give the weight of the clean image $\sqrt{\bar{\alpha}_t}$ and of the noise $\sqrt{1-\bar{\alpha}_t}$. Sketch how an image x_t would look.
(b) The *signal-to-noise ratio* is $\mathrm{SNR}_t = \bar{\alpha}_t/(1-\bar{\alpha}_t)$. Why should SNR_T be close to zero? What goes wrong at sampling time if it is not?

Solution

(a)

t	10	100	250	500	1000
$\sqrt{\bar{\alpha}_t}$	0.999	0.947	0.724	0.280	0.006
$\sqrt{1-\bar{\alpha}_t}$	0.044	0.321	0.690	0.960	1.000

At $t = 10$ the image is essentially clean; at $t = 100$ visibly grainy; around $t = 250$ signal and noise are comparable; at $t = 500$ only coarse shapes are visible; at $t = 1000$ it is pure noise.
(b) $\mathrm{SNR}_{1000} \approx 4 \times 10^{-5}$. Sampling starts from pure noise $x_T \sim \mathcal{N}(0, I)$. This matches training only if x_T contains (almost) no information about x_0. If SNR_T were noticeably positive, training inputs at step T would still contain image information that sampling inputs never have. This train/test mismatch gives poor samples (e.g. limited brightness range).

Exercise 4

Algorithm 1 (Training) of DDPM repeats: sample x_0; sample $t \sim \mathrm{Uniform}\{1, \dots, T\}$; sample $\varepsilon \sim \mathcal{N}(0, I)$; take a gradient step on $\left\|\varepsilon - \varepsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon,\, t)\right\|^2$.

(a) Identify the network input, the target and the loss in this line. How does this correspond to the loss computed in Lab 1A?
(b) The slides contrast training "ideally" (add noise step by step) with "in reality". What is the practical difference, and why does it not change what the network learns?
(c) Why is t sampled uniformly at random in each iteration instead of looping over $t = 1, \dots, T$ for every image?

Solution

(a) Input: the noisy image $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon$ together with t. Target: the noise ε that was actually added. Loss: mean-squared error between predicted and true noise. In Lab 1A this is `F.mse_loss(noise_pred, noise)`, with the noisy image created by the scheduler's `add_noise`.
(b) "Ideally" one would add noise t times; "in reality" the closed form (Exercise 2) produces x_t in one shot. The distribution of x_t given x_0 is identical, so the network sees statistically the same training pairs; only the cost differs.
(c) Every iteration then trains a random noise level, so over many iterations all levels are covered evenly, and each minibatch mixes levels (a less biased gradient). Looping over all 1000 steps per image would make one iteration 1000× more expensive and produce strongly correlated updates.

Exercise 5

DDPM is trained by maximising a lower bound (ELBO) on $\log P_\theta(x_0)$, which splits into three terms: (i) a reconstruction term $\mathbb{E}_q[\log P(x_0|x_1)]$; (ii) $\mathrm{KL}(q(x_T|x_0) \,\|\, P(x_T))$; (iii) a sum over t of $\mathrm{KL}(q(x_{t-1}|x_t, x_0) \,\|\, P_\theta(x_{t-1}|x_t))$.

(a) Why is maximising $\log P_\theta(x_0)$ directly "very difficult"?
(b) Which of the three terms depend on the network parameters? Explain the lecture's remark that term (ii) has "nothing to do with the model".
(c) Both distributions in term (iii) are Gaussians with (fixed) variances. Explain in words why minimising this KL reduces to making the network's mean match the mean of $q(x_{t-1}|x_t, x_0)$, and why this in turn becomes the simple noise-prediction loss.

Solution

(a) $P_\theta(x_0) = \int P(x_T) \prod_t P_\theta(x_{t-1}|x_t)\, dx_{1:T}$ integrates over all possible noise trajectories: an intractable, extremely high-dimensional integral.
(b) Terms (i) and (iii) depend on θ. Term (ii) compares the fixed forward process at step T with the fixed prior $\mathcal{N}(0, I)$; neither has parameters, so it is a constant (close to 0 for a good

schedule).
(c) With variances fixed, the KL between two Gaussians is (up to constants) the squared distance between their means, $\propto \|\mu_q(x_t, x_0) - \mu_\theta(x_t, t)\|^2$. The target mean is a known combination of x_t and x_0, and x_0 can be written using x_t and the noise ε (p. 56). If the network's mean is parameterised the same way with ε_θ in place of ε, the distance between means becomes proportional to $\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$. DDPM drops the t-dependent weight, which gives the training loss of Exercise 4.

Exercise 6

DDIM reuses the U-Net trained for DDPM but samples with fewer steps. With $\eta = 0$ it is deterministic.

(a) Why can DDIM use the *same* trained network as DDPM without retraining?
(b) In the lab you compare 50 and 10 DDIM steps with DDPM (1000 steps). What do you expect to change in image quality and in sampling time?
(c) What does "deterministic" mean here, and why does it make the sampler (approximately) invertible? Why is a DDPM sample not invertible in the same way?

Solution

(a) The network only learned to predict the noise in x_t for any noise level $\bar{\alpha}_t$. That training objective depends only on the marginals $q(x_t|x_0)$, which DDIM keeps unchanged. DDIM changes only how the noise estimate is used to move between (a subset of) noise levels, so the same noise predictor applies.
(b) Time scales with the number of steps: 50 steps $\approx 20\times$ and 10 steps $\approx 100\times$ faster than 1000. 50 DDIM steps usually look close to DDPM quality; with 10 steps images typically lose fine detail or become slightly blurry/over-smoothed, since each step must make a larger jump. (With a fixed seed, DDIM images at different step counts share the same global layout.)
(c) With $\eta = 0$ no fresh noise is added: the same initial noise x_T always gives the same image. Each step is a deterministic function (a discretised ODE), so it can be run backwards to map an image to its latent noise x_T (DDIM inversion), up to discretisation error. DDPM injects new random noise z at every step, so many different noise paths lead to the same image, and a unique noise cannot be recovered from the image.